



Parte I (35% - 70 pontos)

1. ¹⁰**Explique** as principais diferenças entre o paralelismo ao nível da instrução e o paralelismo baseado em fios de execução (*threads*). Indique quais os tipos comuns dependências que impedem a exploração desse paralelismo.

O paralelismo ao nível da instrução consiste na execução de várias instruções numa mesma PU. O paralelismo ao nível de *threads* consiste na execução de blocos de instruções/funções em várias PUs.

Existem dependências de dados quando uma tarefa (instrução / função) precisa do resultado de outra tarefa(s); Existem as dependências de controlo quando a execução de uma tarefa depende do resultado de uma decisão.

2. ¹⁰**Indique** quais são os principais desafios que se colocam à exploração do processamento vetorial nos processadores atuais.

Os três mais importantes são: 1) apenas o código sem dependências de dados pode ser vetorizado; 2) os acessos irregulares à memória (sem localidades espacial); 3) as dependências de controlo.

3. ¹⁰A memória do tipo HBM (*high-bandwidth memory*), tem vindo a ser apontada como uma solução para o problema da *memory-wall*. **Discuta** até que ponto isto será verdade, dando um especial ênfase no contexto das arquiteturas com um elevado número de PUs (*cores*).

A HBM **reduz significativamente** a limitação da largura de banda (atrasa a *memory wall*, **mas não elimina** completamente) em especial em arquiteturas com um elevado número de PUs. Adicionalmente, não resolve a *memory wall* por completo principalmente por não atacar o problema da latência nos acessos à memória

4. ¹⁰**Indique** quais são os principais fatores que impedem que um sistema de computação paralelo atinga o pico de desempenho na execução de aplicações reais.

O desempenho está limitado pela fração não paralelizada do programa, falta de localidade de dados (esgota facilmente a largura de banda da memória disponível), falta de balanceamento de carga e da sobrecarga de comunicação/sincronização.

5. ²⁰**Discuta** as principais diferenças entre os modelos de programação OpenMP, CUDA e MPI em termos da gestão da localidade dos dados.

Uma arquitetura paralela apresenta vários níveis de hierarquia de memória (bancos de memória, caches, etc.). Os dados poderão estar localizados num ou em vários locais (e.g., réplicas de dados).

OpenMP privilegia **simplicidade**, delegando a gestão da localidade dos dados completamente ao hardware e ao sistema operativo.

CUDA **exige uma gestão explícita da localidade para atingir alto desempenho**, quer pela utilização de *shared memory* ou pelos acessos à memória que deverão ser *coalesced*. (localidade espacial entre *threads*)

MPI **força** o programador a pensar explicitamente na colocação dos dados, sendo o modelo que melhor escala, mas também o mais exigente.

6. ¹⁰O modelo de programação CUDA obriga o programador a organizar as *threads* em blocos. **Explique** as consequências desta organização termos de implementação dos programas paralelos.

Organizar *threads* em blocos em CUDA **força o programador a estruturar o problema em unidades independentes** (o hardware pode distribuir livremente esses blocos pelas SMs disponíveis) que podem explorar a *shared memory* e comunicação/sincronização local ao bloco